

# Project 7.2

## Information Extraction from Images

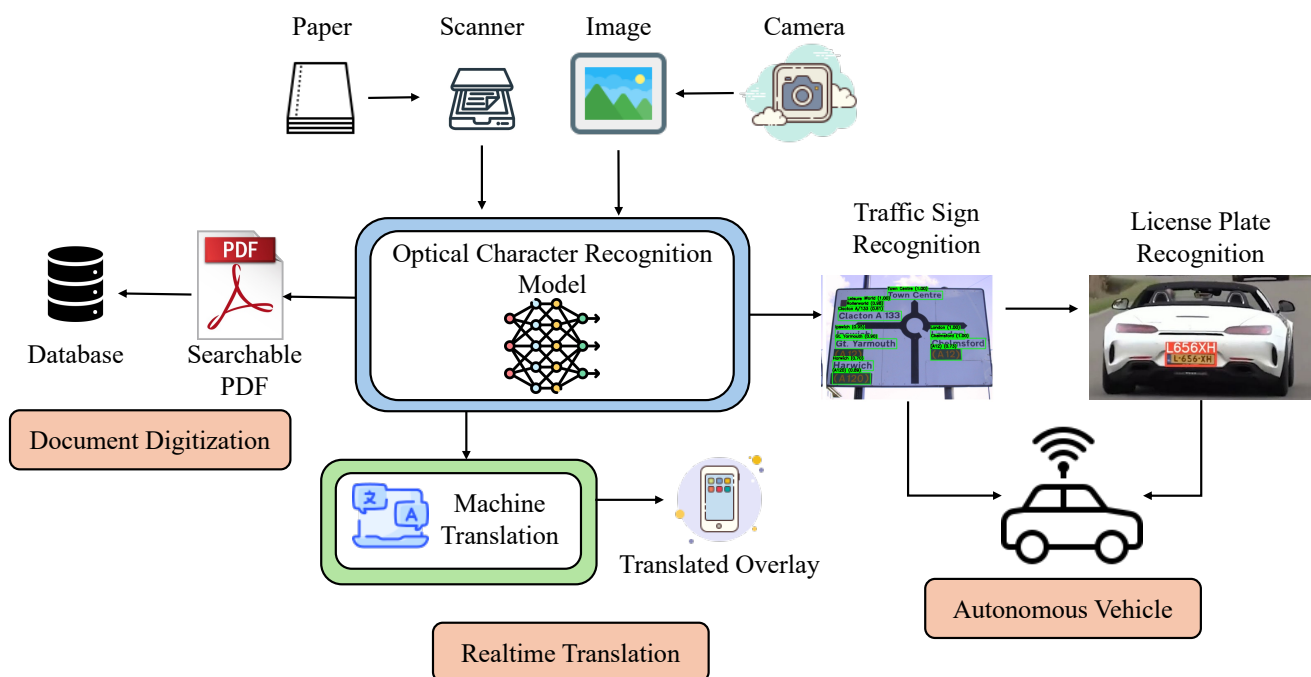
Nguyễn Phúc Thịnh  
Nguyễn Quốc Thái

Hồ Quang Hiển  
Đinh Quang Vinh

### I. Giới thiệu

Trong lĩnh vực thị giác máy tính, bài toán “Optical Character Recognition” (OCR) đóng vai trò là cầu nối quan trọng giữa thế giới thực và không gian số. Mục tiêu cốt lõi của OCR là chuyển đổi hình ảnh chứa văn bản (như tài liệu scan, biển báo giao thông, biển số xe) thành định dạng dữ liệu văn bản mà máy tính có thể xử lý, tìm kiếm và lưu trữ được.

Khác với các bài toán nhận diện vật thể thông thường, văn bản trong hình ảnh mang tính cấu trúc cao và chứa đựng ngữ nghĩa cụ thể. Do đó, việc xây dựng một hệ thống OCR hiệu quả đòi hỏi sự kết hợp chặt chẽ giữa khả năng định vị chính xác và khả năng giải mã chuỗi ký tự.



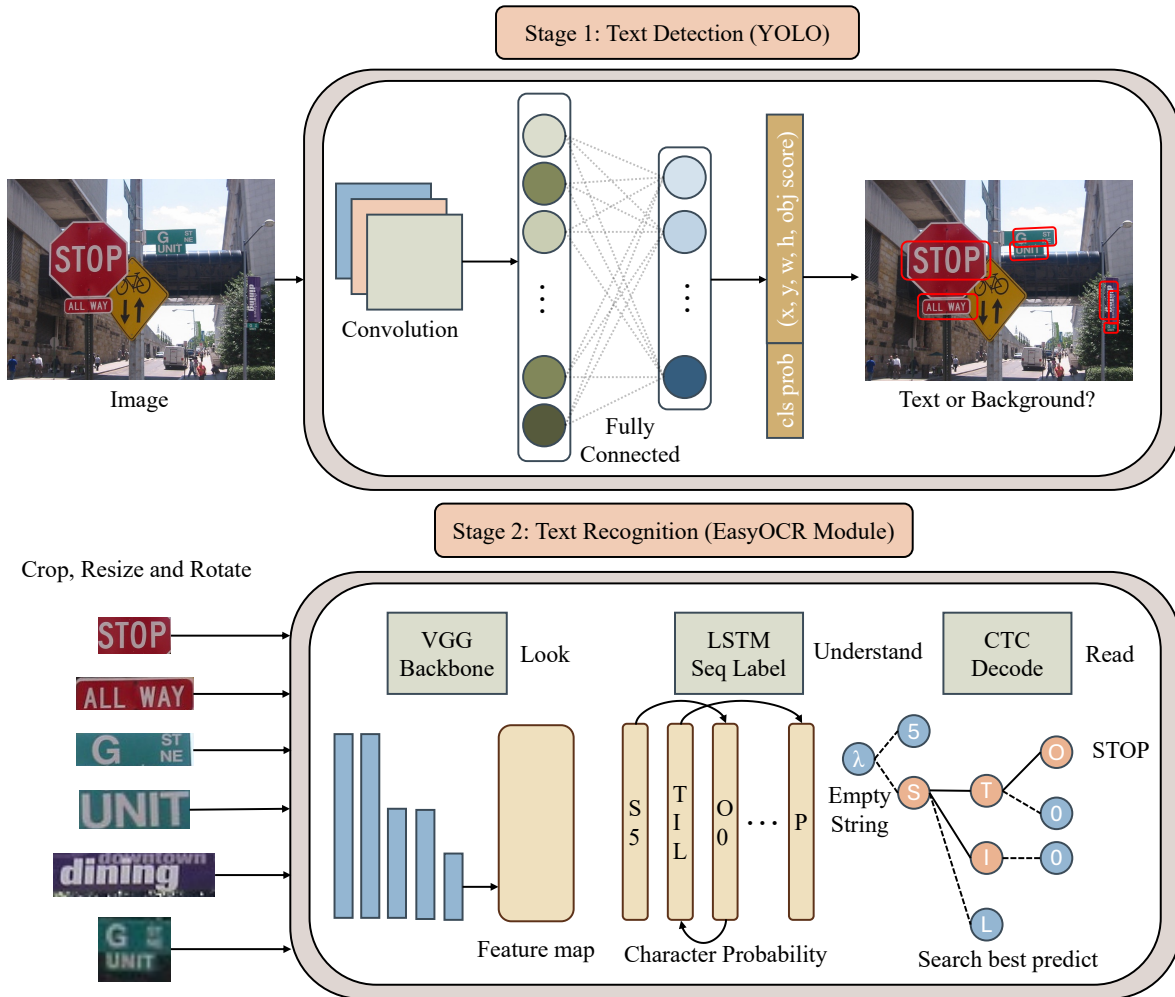
Hình 1: Minh họa các ứng dụng thực tế của OCR: từ số hóa tài liệu, dịch thuật trực tiếp đến xe tự hành.

Trong thực tế, chúng ta thường đối mặt với bài toán “Scene Text Recognition” (nhận diện văn bản trong khung cảnh tự nhiên). Đây là một thách thức lớn hơn nhiều so với nhận diện tài liệu scan truyền thống do các yếu tố nhiễu như: ánh sáng không đồng đều, góc chụp nghiêng, phong chữ đa dạng và nền ảnh phức tạp.

Để giải quyết vấn đề này một cách toàn diện, ta tiếp cận theo mô hình hai giai đoạn. Phương pháp này chia nhỏ bài toán lớn thành hai bài toán con độc lập nhưng bổ trợ lẫn nhau:

1. **Text Detection (Phát hiện văn bản)**: Đây là bước đầu tiên và quan trọng nhất. Mô hình sẽ quét toàn bộ bức ảnh để xác định vị trí của các vùng chứa văn bản, thường được biểu diễn dưới dạng các bounding boxes. Trong bài học này, ta sẽ sử dụng kiến trúc YOLO - một trong những thuật toán phát hiện vật thể tiên tiến nhất hiện nay để đảm nhiệm vai trò này.
2. **Text Recognition (Nhận dạng văn bản)**: Sau khi đã có tọa độ các vùng văn bản, ta tiến hành cắt (crop) các vùng này và đưa vào mô hình nhận dạng để chuyển đổi các điểm ảnh (pixels) thành chuỗi ký tự (string). Module EasyOCR hoặc mô hình CRNN được huấn luyện sẽ được tích hợp ở bước này nhờ khả năng hỗ trợ đa ngôn ngữ và độ chính xác cao.

Việc tách rời hai giai đoạn cho phép ta linh hoạt tối ưu hóa từng thành phần và dễ dàng nâng cấp hệ thống khi có các mô hình mới xuất hiện.



Hình 2: Sơ đồ luồng xử lý dữ liệu: Ảnh đầu vào đi qua mô hình Detection để lấy tọa độ, sau đó qua mô hình Recognition để lấy nội dung văn bản.

Kết thúc bài học này, ta sẽ xây dựng được một hệ thống “End-to-End OCR” hoàn chỉnh, có khả năng nhận đầu vào là một bức ảnh bất kỳ và trả về kết quả là văn bản đã được số hóa cùng vị trí tương ứng của chúng trên ảnh gốc.

# Mục lục

|       |                                                         |    |
|-------|---------------------------------------------------------|----|
| I.    | Giới thiệu . . . . .                                    | 1  |
| II.   | Cơ sở lý thuyết . . . . .                               | 5  |
| II.1. | Text Detection với YOLO . . . . .                       | 5  |
| II.2. | Text Recognition: Kiến trúc CRNN . . . . .              | 6  |
| III.  | Chuẩn bị và tiền xử lý dữ liệu . . . . .                | 7  |
| IV.   | Xây dựng hệ thống OCR sử dụng YOLOv11 và CRNN . . . . . | 9  |
| V.    | Câu hỏi trắc nghiệm . . . . .                           | 20 |
|       | Phụ lục . . . . .                                       | 22 |

## II. Cơ sở lý thuyết

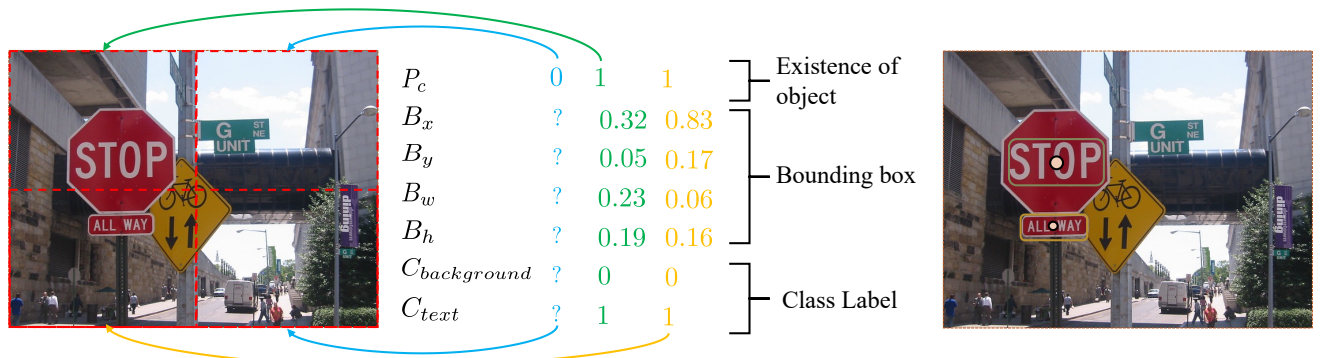
Trước khi đi vào hiện thực hóa, ta cần nắm vững cơ chế hoạt động của hai thành phần chính trong hệ thống: Text Detection và Text Recognition.

### II.1. Text Detection với YOLO

Trong bài toán này, văn bản được coi là một “đối tượng” (object) đặc biệt. YOLO (You Only Look Once) là lựa chọn tối ưu nhờ tốc độ xử lý thời gian thực và khả năng phát hiện tốt các vật thể có kích thước đa dạng.

Cơ chế hoạt động cốt lõi của YOLO dựa trên việc chia bức ảnh đầu vào thành một lưới  $S \times S$ . Mỗi ô lưới sẽ chịu trách nhiệm dự đoán các đối tượng mà tâm của chúng nằm trong ô đó. Với mỗi ô, mô hình sẽ trả về một vector dự đoán bao gồm 3 thành phần chính:

1. Bounding Box Coordinates: Bao gồm 4 giá trị  $(x, y, w, h)$  xác định vị trí và kích thước của văn bản.
2. Objectness Score: Xác suất  $P_{obj}$  cho biết trong khung bao đó có chứa một vật thể bất kỳ hay không (Background vs. Foreground).
3. Class Probabilities: Các giá trị  $(C_{text}, C_{background}, \dots)$  xác định vật thể đó thuộc loại nào.



Hình 3: Kiến trúc mạng YOLO với cơ chế chia lưới ảnh và vector đầu ra tiêu chuẩn.

### Class Probabilities trong bài toán 1 lớp với YOLOv11

Một câu hỏi đặt ra là: Tại sao cần các tham số phân loại lớp (như  $C_{text}$ ) trong khi tham số “Existence of object” ( $P_{obj}$ ) đã đủ để xác định có văn bản hay không?

Với YOLOv11 (Ultralytics), mô hình thường không tách riêng tham số “Objectness” ( $P_{obj}$ ) như một số phiên bản YOLO trước đây. Thay vào đó, điểm tin cậy (confidence/score) của mỗi dự đoán được suy ra trực tiếp từ score của lớp.

Trường hợp 1 lớp (Text): score thường được tính như

$$s = \sigma(z_{text})$$

và được dùng để lọc ngưỡng confidence threshold và NMS.

**Giữ nguyên kiến trúc và pipeline:** Giữ lại  $C_{text}$  và đặt num\_class = 1 giúp giữ nguyên head, loss, NMS, export và toàn bộ codebase chuẩn của mô hình YOLO. Nhờ đó ta fine-tune nhanh, ổn định và dễ mở rộng sang đa lớp về sau mà không phải sửa sâu cấu trúc mạng.

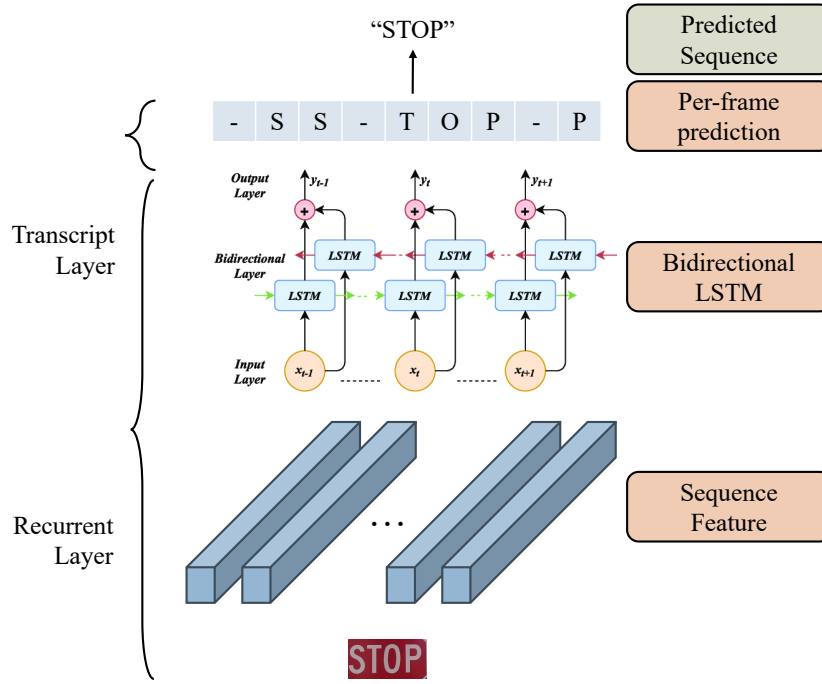
*Dù về mặt lý thuyết có sự dư thừa thông tin, nhưng đây là chiến lược tối ưu để xây dựng pipeline nhanh chóng và ổn định.*

## II.2. Text Recognition: Kiến trúc CRNN

Sau khi YOLO định vị được vùng ảnh chứa chữ, ta cần một mô hình chuyên biệt để giải mã hình ảnh thành văn bản. Kiến trúc CRNN (Convolutional Recurrent Neural Network) là lựa chọn tiêu chuẩn cho tác vụ này nhờ khả năng xử lý chuỗi ký tự có độ dài thay đổi mà không cần chia cắt từng ký tự (segmentation-free).

Mô hình CRNN bao gồm ba thành phần cốt lõi hoạt động tuần tự:

1. **Feature Extraction (Trích xuất đặc trưng):** Sử dụng mạng CNN (như ResNet/VGG) để trích xuất các đặc trưng hình ảnh (feature maps) từ vùng ảnh đầu vào.
2. **Sequence Modeling (Mô hình hóa chuỗi):** Sử dụng mạng RNN (thường là Bi-LSTM) để quét qua các feature maps và nắm bắt mối quan hệ ngữ cảnh giữa các ký tự.
3. **Decoding (Giải mã):** Sử dụng lớp CTC (Connectionist Temporal Classification) để dự đoán chuỗi ký tự đầu ra mà không cần căn chỉnh thời gian chính xác (alignment).



Hình 4: Kiến trúc CRNN với luồng xử lý từ trích xuất đặc trưng đến giải mã ký tự.

Luồng xử lý dữ liệu:

1. Input: Ảnh đầu vào được chuẩn hóa kích thước (thường là  $32 \times W$ ).
2. Conv Layers: Trích xuất chuỗi các vector đặc trưng (Feature Sequence) từ ảnh.
3. Recurrent Layers: Mạng Bi-LSTM dự đoán nhãn dựa trên ngữ cảnh hai chiều.
4. Transcription: Lớp CTC chuyển đổi các dự đoán thành văn bản cuối cùng.

Bản đồ xác suất (Probability Map):

Tại đây, mô hình tạo ra một ma trận xác suất. Các hàng đại diện cho các ký tự trong từ điển, và các cột đại diện cho từng lát cắt thời gian của ảnh.

|     | 1   | 2   | 3   | ... | t   |
|-----|-----|-----|-----|-----|-----|
| a   | 0.1 | 0.9 | 0.0 |     | 0.8 |
| b   | 0.7 | 0.1 | 0.0 |     | 0.3 |
| ... |     |     |     |     |     |
| z   | 0.0 | 0.0 | 0.0 |     | 0.0 |

→ "Banana"

Ma trận này giúp máy tính chọn ra ký tự có xác suất cao nhất tại mỗi vị trí.

Để cân bằng giữa kiến thức nền tảng và tính ứng dụng, notebook này sẽ tiếp cận bài toán theo hai bước:

- Phần 1 - Xây dựng CRNN từ đầu (Implementation from Scratch): Ta sẽ tự định nghĩa các lớp CNN, RNN và CTC Loss. Mục tiêu là giúp bạn hiểu sâu về luồng dữ liệu (data flow) và cách các layer liên kết với nhau bên trong một mô hình OCR.
- Phần 2 - Sử dụng thư viện EasyOCR: Sau khi đã nắm vững nguyên lý, ta sẽ sử dụng EasyOCR - một thư viện mã nguồn mở mạnh mẽ tích hợp sẵn CRNN đã được huấn luyện (pre-trained). Đây là thành phần chính sẽ được ghép nối vào Pipeline cuối cùng để đảm bảo độ chính xác và tốc độ xử lý.

### III. Chuẩn bị và tiền xử lý dữ liệu

Dữ liệu đầu vào của bài toán thường ở định dạng XML (theo chuẩn ICDAR), chứa thông tin tọa độ ( $x_{\min}$ ,  $y_{\min}$ ,  $x_{\max}$ ,  $y_{\max}$ ). Tuy nhiên, YOLO yêu cầu định dạng đầu vào là file .txt với tọa độ đã được chuẩn hóa (normalized) theo tâm ( $x_c$ ,  $y_c$ ,  $w$ ,  $h$ ).

Ta sử dụng công thức sau để chuyển đổi:

$$\begin{cases} x_c = \frac{x_{\min} + x_{\max}}{2 \times W_{img}} \\ y_c = \frac{y_{\min} + y_{\max}}{2 \times H_{img}} \\ w = \frac{x_{\max} - x_{\min}}{W_{img}} \\ h = \frac{y_{\max} - y_{\min}}{H_{img}} \end{cases}$$

Trong đó  $W_{img}, H_{img}$  là chiều rộng và chiều cao của ảnh gốc. Quá trình này được thực hiện thông qua hàm xử lý như sau:

### Chuyển đổi tọa độ sang YOLO format

```

1 def xml_to_yolo_bbox(bbox, w, h):
2     # x_min, x_max, y_min, y_max
3     x_center = ((bbox[2] + bbox[0]) / 2) / w
4     y_center = ((bbox[3] + bbox[1]) / 2) / h
5     width = (bbox[2] - bbox[0]) / w
6     height = (bbox[3] - bbox[1]) / h
7     return [x_center, y_center, width, height]
```

Sau khi chuyển đổi, cấu trúc thư mục dữ liệu cho việc huấn luyện sẽ được tổ chức lại để tương thích với ultralytics:

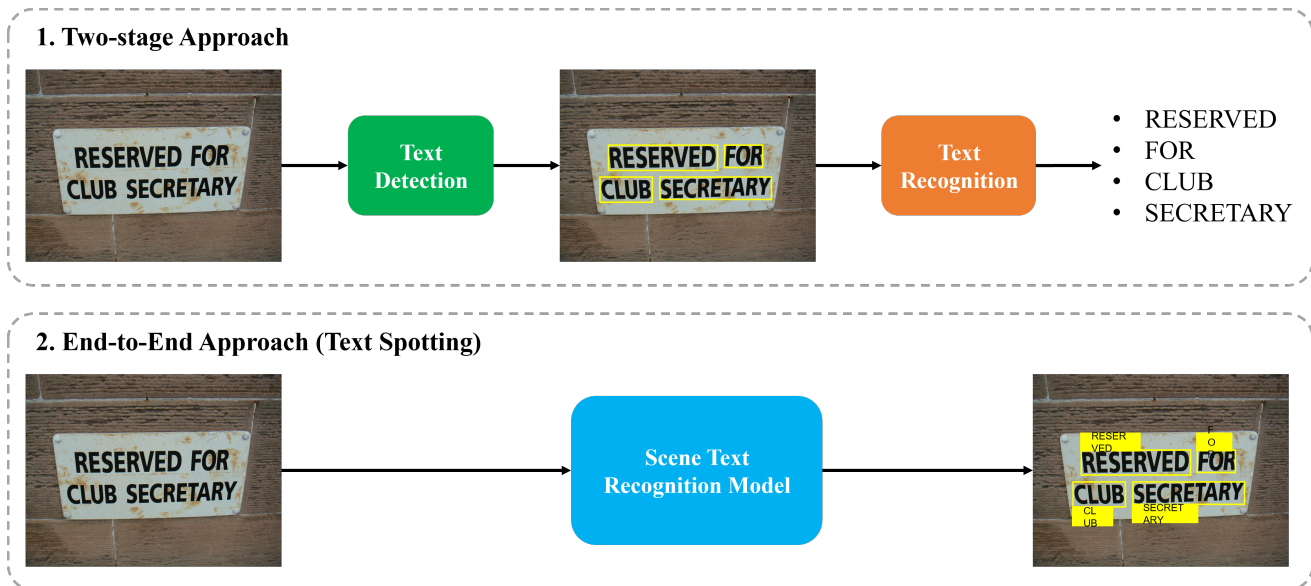
- `images/`: Chứa các file ảnh gốc (.jpg).
- `labels/`: Chứa các file nhãn (.txt) tương ứng với tên ảnh.



## IV. Xây dựng hệ thống OCR sử dụng YOLOv11 và CRNN

Trong bài toán này, chúng ta xây dựng hệ thống OCR gồm hai giai đoạn: dùng **YOLOv11** để phát hiện vùng chứa văn bản (text detection), sau đó dùng **CRNN** để nhận dạng chuỗi ký tự (text recognition) trên các ảnh crop.

Dữ liệu sử dụng là **ICDAR2003**, bao gồm ảnh kèm annotation phục vụ cả phát hiện (bounding box) và nhận dạng (nhãn văn bản) để huấn luyện và đánh giá mô hình.



Hình 5: Minh họa hệ thống OCR được xây dựng.

### Bước 1: Import thư viện và khởi tạo môi trường OCR

```

1 import os
2 import random
3 import shutil
4 import time
5 import xml.etree.ElementTree as ET
6
7 import cv2
8 import matplotlib.pyplot as plt
9 import numpy as np
10 import timm
11 import torch
12 import torch.nn as nn
13 import ultralytics
14 import yaml
  
```

```

15 from PIL import Image
16 from sklearn.model_selection import train_test_split
17 from torch.nn import functional as F
18 from torch.utils.data import DataLoader, Dataset
19 from torchvision import transforms
20 from ultralytics import YOLO
21
22 import json
23 import easyocr
24 from transformers import TrOCRProcessor, VisionEncoderDecoderModel
25 import Levenshtein
26
27 %matplotlib inline
28 ultralytics.checks()

```

Ở bước này, ta import các thư viện cần thiết để xây dựng pipeline OCR gồm *phát hiện vùng chữ* và *nhận dạng văn bản*. Nhóm `os/random/shutil/time` và `xml.etree.ElementTree` hỗ trợ thao tác dữ liệu, chia tách và đọc annotation; trong khi `cv2`, `PIL`, `numpy`, `matplotlib` phục vụ đọc ảnh, tiền xử lý và trực quan hoá khi debug.

Phần mô hình và huấn luyện dùng PyTorch (`torch`, `nn`, `F`, `Dataset/DataLoader`, `transforms`) kết hợp các công cụ mô hình: `ultralytics.YOLO` cho text detection, `timm` cho backbone, `easyocr` làm baseline nhanh, và `TrOCR` cho text recognition. Cuối cùng, `Levenshtein` dùng để đánh giá độ sai khác chuỗi, `json` để lưu/đọc log, `%matplotlib inline` để hiển thị ảnh trong notebook, và `ultralytics.checks()` để kiểm tra môi trường trước khi chạy các bước tiếp theo.

## Bước 2: Chuyển annotation sang định dạng YOLO và tạo file cấu hình dữ liệu

```

1 def convert_to_yolo_format(image_paths, image_sizes, bounding_boxes):
2     yolo_data = []
3
4     for img_path, img_size, bbs in zip(image_paths, image_sizes, bounding_boxes):
5         img_w, img_h = img_size
6         yolo_bbs = []
7
8         for bb in bbs:
9             x, y, w, h = bb
10            x_center = (x + w / 2) / img_w
11            y_center = (y + h / 2) / img_h
12            w_norm = w / img_w
13            h_norm = h / img_h
14
15            yolo_bbs.append([0, x_center, y_center, w_norm, h_norm])
16
17        yolo_data.append((img_path, yolo_bbs))
18
19    return yolo_data

```

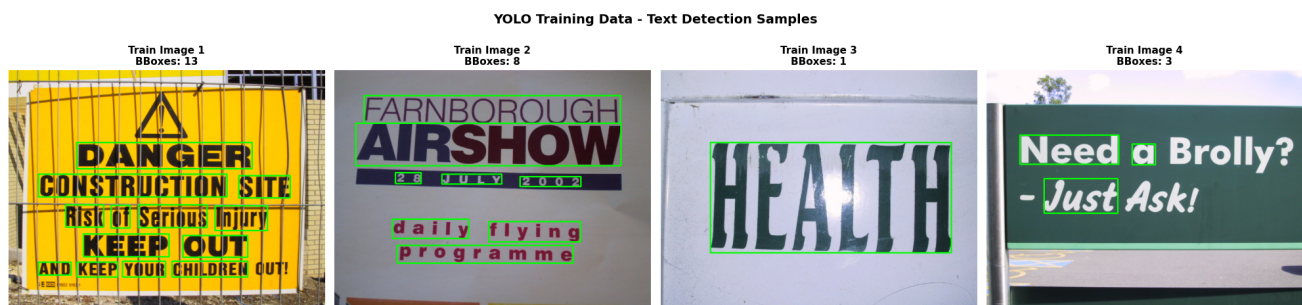
```

20
21
22 yolo_data = convert_to_yolo_format(image_paths, image_sizes, bounding_boxes)
23
24 train_yolo_data, val_yolo_data = train_test_split(
25     yolo_data, test_size=0.2, random_state=42
26 )
27
28 class_labels = ["text"]
29 data_yaml = {
30     "path": os.path.abspath(save_yolo_data_dir),
31     "train": "train/images",
32     "val": "val/images",
33     "nc": 1,
34     "names": class_labels,
35 }
36
37 yolo_yaml_path = os.path.join(save_yolo_data_dir, "data.yaml")
38 with open(yolo_yaml_path, "w") as f:
39     yaml.dump(data_yaml, f, default_flow_style=False)

```

Ở bước này, ta chuyển các bounding box gốc theo pixel sang định dạng YOLO gồm tâm hộp và kích thước hộp, sau đó chuẩn hoá theo chiều rộng và chiều cao ảnh để mô hình học ổn định trên các ảnh có kích thước khác nhau. Vì bài toán chỉ phát hiện vùng chữ, ta dùng một lớp duy nhất với nhãn "text" và gán chỉ số lớp là 0 cho mọi bounding box.

Tiếp theo, ta chia dữ liệu thành train và validation theo tỉ lệ 80/20. Cuối cùng, ta tạo file cấu hình data.yaml, trong đó khai báo thư mục gốc của dữ liệu, đường dẫn ảnh train/val, số lượng lớp và tên lớp, để Ultralytics YOLO có thể đọc đúng cấu trúc dữ liệu khi huấn luyện và đánh giá.



Hình 6: Vài mẫu dữ liệu trong data train text detection.

## Bước 3: Huấn luyện mô hình YOLOv11 cho text detection

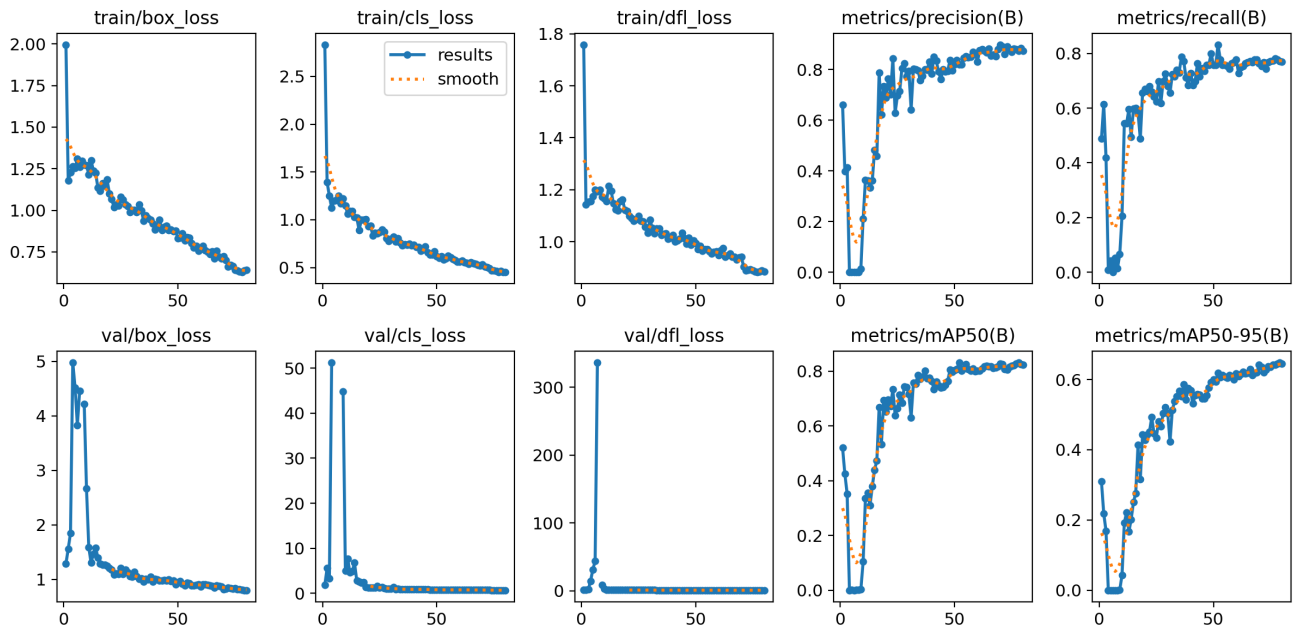
```

1 # Load YOLO model
2 yolo_model = YOLO("yolo11m.pt")
3
4 # Train model
5 yolo_results = yolo_model.train(
6     data=yolo_yaml_path,
7     epochs=80,
8     imgsz=640,
9     cache=True,
10    patience=20,
11    plots=True,
12 )

```

Ở bước này, ta khởi tạo mô hình YOLOv11 từ trọng số tiền huấn luyện "yolo11m.pt" để tăng tốc hội tụ và cải thiện chất lượng phát hiện vùng chữ. Sau đó, ta huấn luyện mô hình với file cấu hình dữ liệu `yolo_yaml_path`, đặt số epoch là 80 và kích thước ảnh đầu vào là 640 để cân bằng giữa độ chính xác và tốc độ.

Tham số `cache` giúp tăng tốc đọc dữ liệu, `patience` hỗ trợ dừng sớm khi mô hình không cải thiện trong một số epoch liên tiếp, và `plots` tự động xuất các biểu đồ theo dõi quá trình huấn luyện. Đầu ra `yolo_results` được lưu lại để phục vụ đánh giá và dùng mô hình detection ở các bước tiếp theo của hệ thống OCR.



Hình 7: Learning Curves của quá trình train YOLOv11.

## Bước 4: Định nghĩa vocabulary và hàm encode, decode cho nhãn văn bản

```

1  # Define vocabulary
2  chars = "0123456789abcdefghijklmnopqrstuvwxyz-"
3  vocab_size = len(chars)
4  blank_char = "-"
5
6  char_to_idx = {char: idx + 1 for idx, char in enumerate(sorted(chars))}
7  idx_to_char = {idx: char for char, idx in char_to_idx.items()}
8
9  max_label_len = max([len(label) for label in labels])
10
11 def encode(text, char_to_idx, max_label_len):
12     encoded = []
13     for char in text:
14         encoded.append(char_to_idx[char])
15
16     label_len = len(encoded)
17
18     # Pad with 0s
19     encoded += [0] * (max_label_len - len(encoded))
20
21     return torch.LongTensor(encoded), torch.tensor(label_len, dtype=torch.long)
22
23
24 def decode(encoded_sequences, idx_to_char, blank_char="-"):
25     decoded_sequences = []
26
27     for seq in encoded_sequences:
28         decoded_label = []
29         prev_char = None
30
31         for token in seq:
32             if token != 0:
33                 char = idx_to_char[token.item()]
34                 if char != blank_char:
35                     if char != prev_char or prev_char == blank_char:
36                         decoded_label.append(char)
37                     prev_char = char
38
39         decoded_sequences.append("".join(decoded_label))
40
41     return decoded_sequences

```

Ở bước này, ta định nghĩa bộ ký tự (vocabulary) cho bài toán recognition gồm chữ số, chữ cái thường và ký tự "-", đồng thời thiết lập hai ánh xạ `char_to_idx` và `idx_to_char` để chuyển đổi giữa ký tự và chỉ số. Ta cũng tính `max_label_len` là độ dài nhãn lớn nhất trong tập dữ liệu để phục vụ việc padding nhãn theo một độ dài cố định khi tạo batch.

Hàm `encode` chuyển một chuỗi nhãn thành tensor chỉ số và trả thêm `label_len` là độ dài thật,

sau đó padding bằng 0 để các nhãn có cùng chiều dài. Ngược lại, hàm `decode` biến các chuỗi chỉ số về lại chuỗi ký tự, bỏ qua token 0 và xử lý ký tự blank để giảm lặp ký tự khi giải mã theo cơ chế CTC.

## Bước 5: Xây dựng Dataset cho bài toán text recognition

```

1 class STRDataset(Dataset):
2     def __init__(
3         self,
4         X,
5         y,
6         char_to_idx,
7         max_label_len,
8         label_encoder=None,
9         transform=None,
10    ):
11        self.transform = transform
12        self.img_paths = X
13        self.labels = y
14        self.char_to_idx = char_to_idx
15        self.max_label_len = max_label_len
16        self.label_encoder = label_encoder
17
18    def __len__(self):
19        return len(self.img_paths)
20
21    def __getitem__(self, idx):
22        label = self.labels[idx]
23        img_path = self.img_paths[idx]
24        img = Image.open(img_path).convert("RGB")
25
26        if self.transform:
27            img = self.transform(img)
28
29        if self.label_encoder:
30            encoded_label, label_len = self.label_encoder(
31                label, self.char_to_idx, self.max_label_len
32            )
33        return img, encoded_label, label_len

```

Ở bước này, ta xây dựng lớp `STRDataset` để phục vụ bài toán text recognition từ các ảnh đã crop vùng chữ. Dataset nhận vào danh sách đường dẫn ảnh `X` và nhãn chuỗi `y`, đồng thời giữ các thông tin cần thiết cho mã hoá nhãn gồm `char_to_idx` và `max_label_len`.

Trong `__getitem__`, mỗi mẫu sẽ đọc ảnh từ `img_path`, chuyển sang RGB và áp dụng `transform` nếu có. Sau đó, nhãn văn bản được mã hoá bằng `label_encoder` để tạo `encoded_label` và `label_len`, và Dataset trả về bộ ba (`img`, `encoded_label`, `label_len`) để dùng trực tiếp trong `DataLoader` và quá trình huấn luyện CRNN với CTC.

Training Data Batch - CRNN Input Samples



Hình 8: Vài mẫu dữ liệu train mô hình CRNN.

## Bước 6: Xây dựng mô hình CRNN cho text recognition

### Code Exercise

```

1 class CRNN(nn.Module):
2     def __init__(
3         self, vocab_size, hidden_size, n_layers, dropout=0.2, unfreeze_layers=3
4     ):
5         super(CRNN, self).__init__()
6
7         backbone = ***Your Code Here***
8         modules = list(backbone.children())[:-2]
9         modules.append(nn.AdaptiveAvgPool2d((1, None)))
10        self.backbone = nn.Sequential(*modules)
11
12        # Unfreeze the last few layers
13        for parameter in self.backbone[-unfreeze_layers:].parameters():
14            parameter.requires_grad = True
15
16        # ResNet34 outputs 512 channels
17        self.mapSeq = nn.Sequential(
18            nn.Linear(512, 512), nn.ReLU(), nn.Dropout(dropout)
19        )
20
21        self.gru = nn.GRU(
22            512,
23            hidden_size,
24            n_layers,
25            bidirectional=True,
26            batch_first=True,
27            dropout=dropout if n_layers > 1 else 0,
28        )
29        self.layer_norm = nn.LayerNorm(hidden_size * 2)
30
31        self.out = nn.Sequential(

```

```

32         ***Your Code Here***
33     )
34
35     def forward(self, x):
36         x = ***Your Code Here***
37         x = x.permute(0, 3, 1, 2)
38         x = x.view(x.size(0), x.size(1), -1) # Flatten the feature map
39         x = ***Your Code Here***
40         x, _ = ***Your Code Here***
41         x = self.layer_norm(x)
42         x = ***Your Code Here***
43         x = x.permute(1, 0, 2) # Based on CTC
44
45     return x

```

Ở bước này, bạn hoàn thiện mô hình CRNN theo pipeline CNN-RNN-CTC để nhận dạng chuỗi ký tự từ ảnh chữ. Trước hết, hãy khởi tạo backbone CNN (ResNet34) với đầu vào 1 kênh và dùng trọng số pretrained để trích xuất feature map, sau đó giữ lại phần encoder và thêm lớp pooling để gom chiều cao về 1, còn chiều rộng được giữ như trục thời gian của chuỗi.

Tiếp theo, hãy hoàn thiện phần head của mô hình: ánh xạ đặc trưng theo từng bước thời gian qua `mapSeq`, đưa chuỗi đặc trưng qua GRU hai chiều để học ngữ cảnh, rồi qua lớp `out` để tạo phân phối xác suất trên vocabulary theo từng bước thời gian (dạng log-probability phục vụ CTC). Trong `forward`, bạn cần gọi backbone để lấy đặc trưng, chuyển đổi tensor về dạng chuỗi theo chiều rộng, lần lượt đi qua `mapSeq`, GRU, và `out`, cuối cùng hoán vị đầu ra về định dạng CTC yêu cầu trước khi trả về.

## Bước 7: Thiết lập loss, optimizer, scheduler và huấn luyện CRNN

```

1 epochs = 80
2 lr = 1e-3
3 weight_decay = 1e-5
4 scheduler_step_size = epochs * 0.5
5
6 criterion = nn.CTCLoss(
7     blank=char_to_idx[blank_char],
8     zero_infinity=True,
9     reduction="mean",
10 )
11 optimizer = torch.optim.Adam(
12     crnn_model.parameters(),
13     lr=lr,
14     weight_decay=weight_decay,
15 )
16 scheduler = torch.optim.lr_scheduler.StepLR(
17     optimizer, step_size=scheduler_step_size, gamma=0.1

```



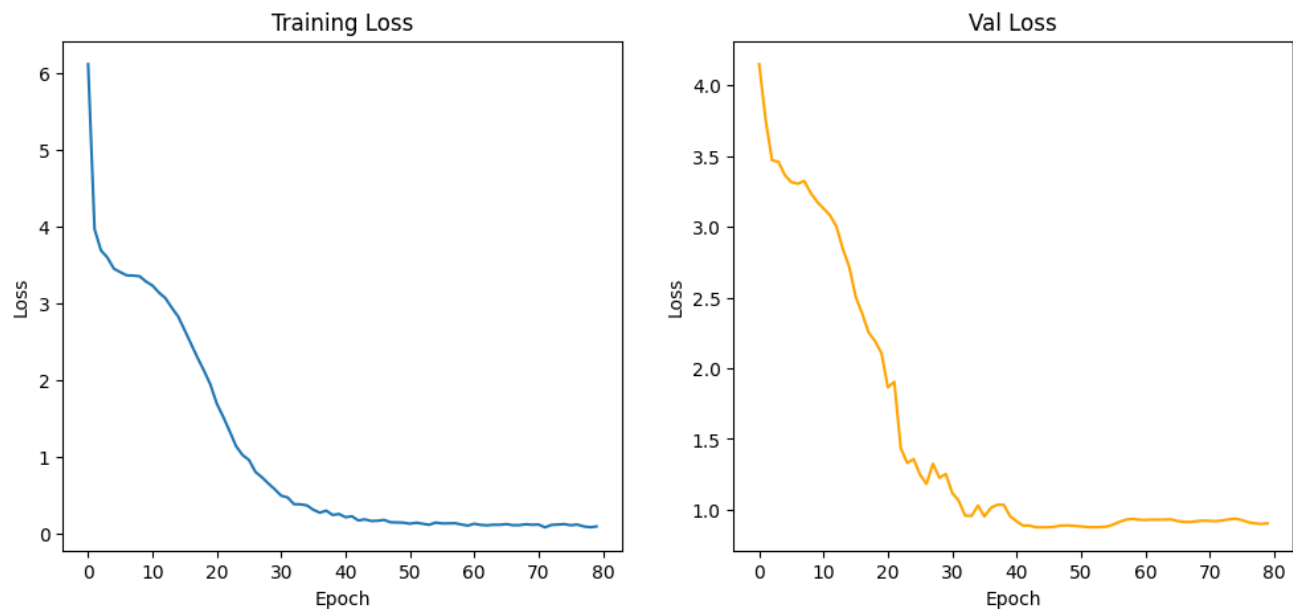
```

18 )
19
20 train_losses, val_losses = fit(
21     crnn_model,
22     train_loader,
23     val_loader,
24     criterion,
25     optimizer,
26     scheduler,
27     device,
28     epochs,
29 )

```

Ở bước này, ta thiết lập quá trình huấn luyện cho mô hình CRNN bằng hàm mất mát CTC (`nn.CTCLoss`) với `blank` là ký tự rỗng trong vocabulary, đồng thời bật `zero_infinity` để tránh lỗi số học trong một số trường hợp đặc biệt. Mô hình được tối ưu bằng Adam với `lr = 1e-3` và `weight_decay = 1e-5` nhằm giúp học ổn định và hạn chế overfitting.

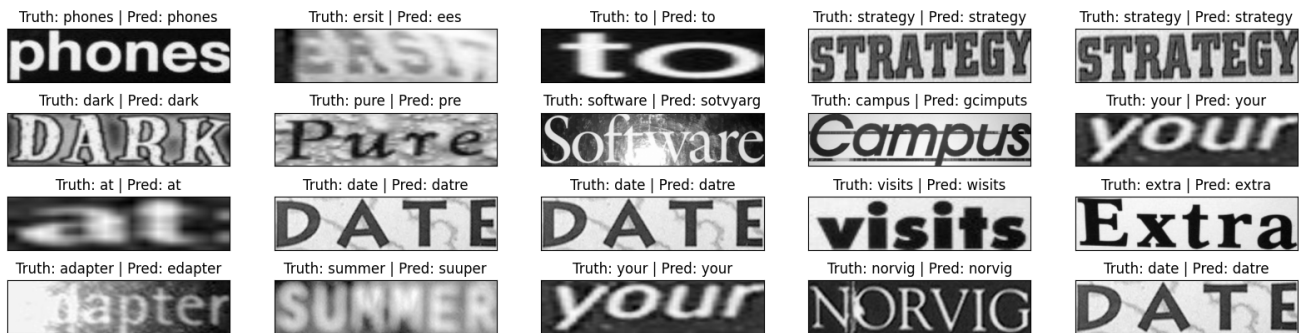
Ta sử dụng StepLR để giảm learning rate theo chu kỳ, với `step_size` bằng một nửa tổng số epoch và `gamma = 0.1`. Cuối cùng, ta gọi hàm `fit` để train trên `train_loader` và đánh giá trên `val_loader`, thu về `train_losses` và `val_losses`. Phần cài đặt chi tiết của hàm `fit` được trình bày ở Phụ lục để tránh làm đoạn code chính quá dài.



Hình 9: Biểu đồ loss của quá trình training và validation.

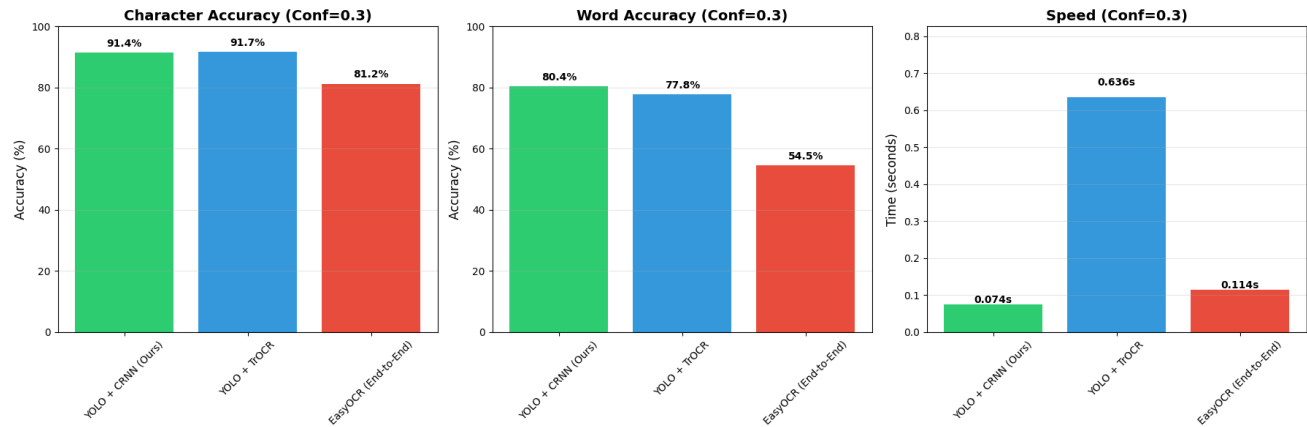
```
1 # View samples from test set
2 sample_result = []
3
4 for i in range(20):
5     idx = np.random.randint(len(test_dataset))
6     img, label, label_len = test_dataset[idx]
7     img = img.to(device)
8     label = label.to(device)
9     label = decode_label([label], idx_to_char)[0]
10    logits = crnn_model(img.unsqueeze(0))
11
12    pred_text = decode(logits.permute(1, 0, 2).argmax(2), idx_to_char)[0]
13
14    sample_result.append((img, label, pred_text))
```

Tiếp theo, ta chuyển logits về dạng phù hợp và lấy lớp có xác suất lớn nhất theo từng bước thời gian, rồi dùng hàm `decode` để chuyển chuỗi chỉ số dự đoán thành văn bản. Cuối cùng, ta lưu lại bộ ba gồm ảnh, nhãn thật và văn bản dự đoán vào `sample_result` để phục vụ hiển thị trực quan và đối chiếu kết quả.



Hình 10: Kết quả inference của hệ thống OCR.

Ngoài ra, để đánh giá toàn diện hơn hệ thống OCR, chúng tôi so sánh ba phương pháp gồm YOLO + CRNN (Ours), YOLO + TrOCR và EasyOCR theo ba tiêu chí: Character Accuracy, Word Accuracy và thời gian suy luận tại ngưỡng Conf=0.3.



Hình 11: Đánh giá so sánh các chỉ số của 3 phương pháp OCR.

Kết quả trong Hình cho thấy hệ YOLO + CRNN (Ours) đạt hiệu năng cân bằng tốt nhất giữa độ chính xác và tốc độ khi so với YOLO + TrOCR và EasyOCR. Về Character Accuracy (Conf=0.3), mô hình của chúng tôi đạt 91.4%, gần tương đương YOLO + TrOCR (91.7%) và cao hơn rõ rệt so với EasyOCR (81.2%). Về Word Accuracy, YOLO + CRNN đạt 80.4%, nhỉnh hơn YOLO + TrOCR (77.8%) và vượt xa EasyOCR (54.5%). Đặc biệt, về tốc độ suy luận, YOLO + CRNN nhanh nhất với 0.074s, nhanh hơn nhiều so với YOLO + TrOCR (0.636s) và cũng nhanh hơn EasyOCR (0.114s).



Hình 12: Visualize kết quả chạy của 3 phương pháp.

Lưu ý rằng ba phương pháp trên không được huấn luyện theo cùng một thiết lập (cùng dữ liệu, cùng chiến lược fine-tune và cùng hyperparameter), nên phép so sánh này mang tính tham khảo và có phần chủ quan. Ngoài ra, **EasyOCR** là một hệ OCR end-to-end phổ biến, thường tối ưu theo hướng nhận dạng chuỗi/câu và được cộng đồng sử dụng rộng rãi nhờ khả năng cân bằng tương đối tốt giữa độ chính xác và tốc độ trong nhiều kịch bản thực tế.

## V. Câu hỏi trắc nghiệm

- (Lý thuyết – Hai hướng tiếp cận OCR) Điểm khác nhau cốt lõi giữa OCR theo hướng Detection–Recognition và OCR End-to-End là gì?
  - Detection–Recognition chỉ dùng cho chữ viết tay, còn End-to-End chỉ dùng cho chữ in.
  - Detection–Recognition tách hai giai đoạn phát hiện vùng chữ và nhận dạng, còn End-to-End dự đoán trực tiếp văn bản từ ảnh đầu vào theo một pipeline thống nhất.
  - Detection–Recognition không cần dữ liệu nhãn, còn End-to-End bắt buộc có nhãn.
  - Detection–Recognition luôn nhanh hơn End-to-End trong mọi trường hợp.
- (Lý thuyết – Vai trò của YOLOv11) Trong hệ thống YOLOv11 + CRNN, YOLOv11 được dùng để làm gì?
  - Mã hoá nhãn văn bản thành chỉ số để huấn luyện.
  - Phát hiện bounding box của vùng chữ để crop và đưa sang mô hình nhận dạng.
  - Giải mã chuỗi ký tự dự đoán theo cơ chế CTC.
  - Chuẩn hoá ảnh về 1 kênh grayscale trước khi train.
- (Lý thuyết – Vai trò của CRNN) Trong hệ thống YOLOv11 + CRNN, CRNN được dùng để làm gì?
  - Dự đoán bounding box của vùng chữ trong ảnh gốc.
  - Tạo file cấu hình dữ liệu `data.yml` cho YOLO.
  - Nhận dạng chuỗi ký tự từ ảnh đã crop vùng chữ.
  - Lọc bỏ các ký tự đặc biệt trong annotation XML.
- (Lý thuyết – Lợi ích của tách giai đoạn) Ưu điểm phổ biến của hướng Detection–Recognition so với End-to-End là gì?
  - Dễ kiểm soát và thay thế từng module (detector hoặc recognizer) theo yêu cầu bài toán.
  - Không cần nhãn bounding box, chỉ cần nhãn chuỗi văn bản.
  - Luôn cho độ chính xác cao hơn trong mọi bộ dữ liệu.
  - Không cần tiền xử lý ảnh vì mô hình tự xử lý hết.
- (Lý thuyết – Hạn chế của tách giai đoạn) Nhược điểm thường gặp của hướng Detection–Recognition là gì?
  - Không thể xử lý ảnh có nhiều dòng chữ.
  - Lỗi của detection có thể lan truyền sang recognition do crop sai hoặc thiếu vùng chữ.
  - Không thể dùng mô hình pretrained cho backbone.

- (d) Không thể chạy trên GPU.
6. (Lý thuyết – CTC Loss) Vì sao CRNN thường dùng CTC Loss trong bài toán nhận dạng chuỗi?
- (a) Vì CTC yêu cầu nhãn phải có cùng chiều dài với output theo thời gian.
  - (b) Vì CTC cho phép huấn luyện mà không cần căn chỉnh ký tự theo từng bước thời gian giữa dự đoán và nhãn thật.
  - (c) Vì CTC tối ưu trực tiếp IoU của bounding box.
  - (d) Vì CTC thay thế hoàn toàn nhu cầu dùng vocabulary và encode/decode.
7. (Lý thuyết – Blank token) Vai trò của ký tự blank trong cơ chế CTC là gì?
- (a) Thể hiện khoảng trống giữa các từ và luôn xuất hiện trong nhãn thật.
  - (b) Là ký tự rỗng giúp mô hình biểu diễn các bước thời gian không phát ra ký tự hữu ích khi căn chỉnh chuỗi.
  - (c) Là ký tự dùng để thay thế mọi ký tự không có trong vocabulary.
  - (d) Là ký tự bắt buộc phải được dự đoán ở cuối mỗi chuỗi.
8. (Lý thuyết – CNN trước RNN) Lý do chính khi dùng CNN (ResNet34) trước RNN (GRU) trong CRNN là gì?
- (a) CNN giúp biến ảnh thành chuỗi đặc trưng theo chiều rộng trước khi RNN học phụ thuộc ngữ cảnh theo chuỗi.
  - (b) CNN dùng để dự đoán trực tiếp văn bản nên không cần RNN.
  - (c) RNN chỉ hoạt động được khi ảnh có 3 kênh RGB.
  - (d) CNN làm tăng số ký tự trong vocabulary để nhận dạng tốt hơn.
9. (Lý thuyết – Augmentation cho recognition) Mục tiêu của RandomPerspective và RandomRotation trong transform train cho recognition là gì?
- (a) Tăng số lớp của bài toán bằng cách tạo thêm ký tự mới.
  - (b) Tăng khả năng tổng quát khi chữ bị nghiêng hoặc biến dạng nhẹ trong ảnh thực tế.
  - (c) Giảm chiều rộng ảnh để CTC chạy nhanh hơn.
  - (d) Giúp mô hình học đúng thứ tự thư mục dữ liệu train/val.
10. (Lý thuyết – So sánh thực nghiệm) Khi so sánh các phương pháp OCR nhưng không huấn luyện theo cùng một thiết lập, nhận định nào phù hợp nhất?
- (a) Kết quả so sánh vẫn hoàn toàn khách quan và có thể kết luận chắc chắn mô hình nào tốt nhất.
  - (b) Kết quả chỉ mang tính tham khảo vì có thể bị ảnh hưởng bởi dữ liệu, chiến lược fine-tune và hyperparameter khác nhau.
  - (c) Không thể đánh giá bất kỳ chỉ số nào nếu không train cùng thiết lập.
  - (d) Chỉ cần so sánh tốc độ là đủ, không cần so sánh độ chính xác.

# Phụ lục

- Hint:** Các file code gợi ý và dữ liệu nếu có được lưu trong thư mục có thể được tải [tại đây](#).
- Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải [tại đây](#) (**Lưu ý:** Sáng thứ 3 khi hết deadline phần project, ad mới copy các nội dung bài giải nêu trên vào đường dẫn).
- Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 giờ.

## AIO\_QAs-Verified

🔒 Nhóm Riêng tư · 1,4K thành viên



Hình 13: Hình ảnh group facebook AIO Q&A

### 4. Rubric:

| Phần | Kiến Thức                                                                                                                                                     | Đánh Giá                                                                                                                                                             |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <ul style="list-style-type: none"> <li>- Hiểu bài toán OCR và pipeline Detection–Recognition.</li> <li>- Vai trò của YOLOv11 trong Text Detection.</li> </ul> | <ul style="list-style-type: none"> <li>- Trình bày đúng quy trình OCR hai giai đoạn.</li> <li>- Chuẩn bị dữ liệu và huấn luyện YOLO đúng cách.</li> </ul>            |
| 2    | <ul style="list-style-type: none"> <li>- Kiến trúc CRNN: CNN (ResNet) – RNN – CTC.</li> <li>- Encode/decode và CTC Loss cho nhận dạng chuỗi.</li> </ul>       | <ul style="list-style-type: none"> <li>- Xây dựng và huấn luyện CRNN cho Text Recognition.</li> <li>- Giải thích được vai trò từng thành phần trong CRNN.</li> </ul> |
| 3    | <ul style="list-style-type: none"> <li>- Tích hợp YOLOv11 + CRNN thành hệ OCR end-to-end.</li> <li>- Các chỉ số: Character/Word Accuracy, Speed.</li> </ul>   | <ul style="list-style-type: none"> <li>- Chạy inference và phân tích kết quả trên tập test.</li> <li>- So sánh với EasyOCR hoặc TrOCR và rút ra nhận xét.</li> </ul> |